Extending Kaleidoscope: `int` and `bool` types, `/` and `>`

CS24BTECH11030 CS24BTECH11036

1 Starting point

Our base is the Kaleidoscope chapter-3 front end (LLVM IR generation, no JIT), ported to **flex/bison**: `lexer.l` replaces `gettok()`, `parser.y` replaces the recursive-descent parser and the `BinopPrecedence` table (now `%left` declarations), `ast.h` holds the AST classes shared by both translation units, and `toy.cpp` holds code generation and the driver. Build with `make`; run with `./toy < tests.kal`. All changes below are relative to that unmodified port (git commit `388e72b`); together they touch 4 source files (+341/−88 lines), mostly `toy.cpp`.

2 Question 1: division (/) and greater-than (>)

- **Lexer:** no change needed, because single characters fall through the catch-all rule `. { return yytext[0]; }`, as in the original `gettok()`.
- **Parser:** new rules `expr '/' expr` and `expr '>' expr`; precedence `%left '<' '>'` (lowest) < `%left '+' '-'` < `%left '*' '/'`, all left-associative. Bison reports no conflicts.
- **Codegen:** `int`: `sdiv`, `icmp sgt/slt` (*signed*, since `int` is signed); `double`: `fdiv`, `fcmp ogt/olt`. We chose *ordered* compares (false if either operand is NaN, as in C) instead of the tutorial's unordered `ult`.
- **Design choice:** a comparison always yields `bool` (`i1`). The tutorial's `uitofp` back to `double` was removed.
- **Semantic check:** an integer division whose divisor is the constant 0 is rejected at compile time, using `dyn_cast<ConstantInt>`.

3 Question 2: `int` and `bool` types

3.1 Syntax (lexer + parser)

- Keywords `int`, `double`, `bool` and literals `true`, `false`. Flex prefers the longest match and then the earliest rule, so `int` is a keyword while `integer` remains an identifier.
- Literals: `12` is an `int` (parsed with `strtol`; an out-of-range value is reported and the lexer returns `YYerror`, so the statement is discarded); `1.5`, `1.` and `.5` are `double`.
- **Every variable definition is typed.** In this chapter, function parameters are the only variable definitions (there is no `var`/assignment), so each parameter carries a type. Functions also declare an *explicit return type*:

```
prototype ::= type id '(' (type id)* ')'    def int add(int a int b) a + b;
type      ::= int | double | bool            extern double sin(double x);
expr      ::= ... | type '(' expr ')'        def int trunc(double x) int(x);
```

The cast rule starts with a keyword token, so it cannot clash with calls (`id '(' args ')'`) or parentheses, and bison still reports no conflicts. Top-level expressions remain anonymous `double` functions; their value is wrapped in an explicit `double(...)` cast, so `4 > 3;` yields `1.0`.

3.2 Type representation and the symbol table

`ast.h` gains `enum class VarType {Double, Int, Bool}`, a `Param {VarType Ty; std::string Name;}` for prototype parameters (it replaces the earlier stringly-typed `vector<vector<string>>`), a `VarType RetTy` in `PrototypeAST`, a new `CastExprAST` node, and `toLLVMType()`: `double`→`double`, `int`→`i32`, `bool`→`i1`. The symbol table `NamedValues` changes from `map<string, Value*>` to `map<string, SymbolInfo>`, where `SymbolInfo = {Value *Val; VarType Ty;}`. It is filled from the prototype's declared parameter types. Lookup uses `find()` instead of `operator[]`, which used to silently insert unknown names.

3.3 Type checking

Expression types are read back from the generated LLVM value (`isDoubleTy()`, `isIntegerTy(32)`, `isIntegerTy(1)`). The rules are:

Construct	Allowed operand types	Result / action
`+ - * /`	int,int | double,double | int,double (int promoted)	operand type
`< >`	same as above	`bool`
any binary op	either operand `bool`	error: "operands cannot be bool"
call `f(`e_1`,..)`	each e_i must equal / implicitly convert to param i	else error naming the argument
function body	must equal / implicitly convert to the declared return type	else error

3.4 Type casting

A single helper, `castTo(V, Target, Explicit, Ctx)`, is used by binary operators, call arguments, `return` values and `CastExprAST`. Only the widening `int`→`double` conversion is implicit. Everything else needs the explicit `T(e)` syntax, so `bool` never leaks into arithmetic by accident:

From → To	Implicit?	Explicit	LLVM instruction
int → double	yes	yes	`sitofp`
double → int	no (narrowing)	yes	`fptosi` (truncates toward 0)
bool → int / double	no	yes	`zext` / `uitofp` (true → 1, not −1)
int / double → bool	no	yes	`icmp ne 0` / `fcmp une 0.0` (NaN is true, as in C)

3.5 Other semantic checks added

- Duplicate parameter names: `def int f(int a double a);` reports "redefinition of parameter 'a'".
- Extern/definition consistency: a definition (or another `extern`) whose signature differs from an earlier declaration is rejected. Before this, LLVM silently created a renamed `g.1`. A definition whose body fails to type-check now keeps the earlier `extern` instead of erasing it.
- `verifyFunction(*F, &errs())` is now checked. Before this change, invalid IR such as `fadd i1` or a `double` function returning `i32` was printed silently. In our test suite the verifier never fires, because every bad case is caught by our own type checks first.
- Error messages now name the variable, function, argument position and types involved.

4 Summary of edits per file

File	Changes relative to the original port
`lexer.l`	`int`/`double`/`bool`/`true`/`false` tokens; the single `NUMBER` token is split into `INTEGER` (`strtol`, range check) and `DOUBLE` (including the `.5` form).
`parser.y`	`%union` gains `VarType ty`, `bool cond`, `int`, `vector<Param>*`; new `type` rule; typed `params`; a return type on `prototype`; the `/`, `>` and cast rules; updated precedence.
`ast.h`	`VarType`, `Param`, `toLLVMType()`; typed literals in `NumberExprAST`; new `CastExprAST`; `RetTy` and getters in `PrototypeAST`.
`toy.cpp`	`SymbolInfo` symbol table; `castTo()`; typed `BinaryExprAST`/`CallExprAST`/return codegen; prototype signature and duplicate-parameter checks; checked `verifyFunction`; anonymous expressions cast to `double`.
`tests.kal`	New file: regression inputs for every rule in this report.

5 Testing

`tests.kal` (in the directory) contains 32 cases covering every rule above. Each case either produces valid IR or one of our own error messages. Examples:

```
def double mix(int a double b) a + b;  ->  %casttmp = sitofp i32 %a to double
                                            %addtmp  = fadd double %casttmp, %b
def bool neg(int a) (a - 5) < 0;       ->  %cmptmp = icmp slt i32 %subtmp, 0 ; ret i1
def int bad1(bool a bool b) a + b;     ->  Error: operator '+': operands cannot be bool (got bool and bool)
def int bad2(double x) x;              ->  Error: return value of 'bad2': cannot implicitly convert double to int
idiv(7, 2.5);                          ->  Error: argument 2 of 'idiv': cannot implicitly convert double to int
extern int g(int x); def double g(double x) x;  ->  Error: conflicting types for 'g'
```

Limitations. There is no unary minus (write `0 - x`), comparing two `bool`s is rejected rather than defined, and top-level expressions always evaluate to `double`.